

MÓDULO 8 — SCRIPTING EN BASH

(AUTOMATIZACIÓN EN LINUX)

8.1. ¿Qué es un script en Bash?

Un **script** es un archivo de texto que contiene comandos que el sistema ejecuta de forma automática, como si los escribieras uno a uno en la terminal.

Son esenciales para:

- Automatizar tareas repetitivas
- Realizar mantenimiento
- Monitorizar sistemas
- Gestionar backups
- Crear herramientas propias

Un script Bash empieza siempre con la línea:

```
#!/bin/bash
```

8.2. Crear y ejecutar un script

1. Crear archivo:

```
nano mi_script.sh
```

2. Añadir:

```
#!/bin/bash
```

```
echo "Hola, mundo"
```

3. Hacerlo ejecutable:

```
chmod +x mi_script.sh
```

4. Ejecutarlo:

```
./mi_script.sh
```

8.3. Variables en Bash

Definir variable:

```
nombre="Alberto"
```

Llamarla:

```
echo $nombre
```

Variables especiales:

- \$0 → nombre del script
 - \$1 \$2 \$3 → argumentos
 - \$# → cantidad de argumentos
 - \$@ → todos los argumentos
-

8.4. Entrada de usuario (read)

```
read -p "Introduce tu nombre: " nombre
```

```
echo "Hola $nombre"
```

8.5. Condicionales (if)

Ejemplo:

```
if [ $edad -ge 18 ]; then
```

```
    echo "Eres mayor de edad"
```

```
else
```

```
    echo "Eres menor de edad"
```

```
fi
```

8.6. Bucles (for, while)

Bucle for:

```
for i in {1..5}; do
```

```
    echo "Número $i"
```

```
done
```

Bucle while:

```
contador=1
```

```
while [ $contador -le 5 ]; do
    echo $contador
    ((contador++))
done
```

8.7. Funciones en Bash

```
saludar() {
    echo "Hola $1"
}
saludar "alumno"
```

8.8. Control de errores

```
comando || echo "El comando falló"
```

8.9. Logs dentro de scripts

```
echo "Backup completado $(date)" >> /var/log/backup.log
```

8.10. Automatización con cron

Editar tareas programadas:

```
crontab -e
```

Formato:

```
* * * * * comando
```

Ejemplo: ejecutar script todos los días a las 22:00:

```
0 22 * * * /home/user/backup.sh
```

36 EJERCICIOS PRÁCTICOS DEL MÓDULO 8

Divididos en básicos, intermedios, avanzados y un mini-proyecto.

✓ EJERCICIOS BÁSICOS (1-12)

1. Crear un script que muestre tu nombre

Mostrar:

echo "Mi nombre es ..."

```
(kali㉿kali)-[~]  
$ echo "Mi nombre es Legui"  
Mi nombre es Legui
```

2. Script que muestre la fecha actual

date

```
(kali㉿kali)-[~]  
$ date  
Thu Jan 22 03:40:58 AM EST 2026
```

3. Script que muestre el directorio actual

pwd

```
(kali㉿kali)-[~]  
$ pwd  
/home/kali
```

4. Script que reciba un argumento y lo muestre

Ejemplo:

./script.sh hola

```
(kali㉿kali)-[~]  
$ ./script.sh  
hola
```

5. Script que cuente del 1 al 10 (for)

```
(kali㉿kali)-[~]  
$ ./script.sh  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

```
#!/bin/bash

for x in {1..10}; do
    echo $x
done
```

6. Script que pida al usuario su nombre y lo salude

```
#!/bin/bash

read -p "Ingresa tu nombre: " nombre
echo "Hola, $nombre!"
```

```
(kali㉿kali)-[~]
$ ./script.sh
Ingresa tu nombre: Legui
Hola, Legui!
```

7. Script que muestre el contenido de /etc

```
GNU nano 8.6
#!/bin/bash

ls /etc
```

```
(kali㉿kali)-[~]
$ ./script.sh
adduser.conf          fonts
alsa                  freeradius
alternatives          freetds
```

8. Script que muestre los 5 procesos que más CPU consumen

Usar ps aux.

```
(kali㉿kali)-[~/Desktop]
$ ./ej.sh
Top 5 procesos por uso de CPU:
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         883   2.6   7.6 447148 154240 tty7    Ssl+  02:27   5:25 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/l
ightdm/root/:0 -nolisten tcp vt7 -novtswitch
kali     10174   0.5   2.9 646356 59388 ?        Sl     05:52   0:00 /usr/bin/qterminal
kali     13444   0.4   5.0 893156 101900 ?        Sl     02:27   0:58 xfwm4
kali     1188    0.4   0.1 215388  2860 ?        Sl     02:27   0:52 /usr/bin/VBoxClient --draganddrop
kali     10182   0.3   0.3 14092  8052 pts/1    Ss     05:52   0:00 /usr/bin/zsh
```

9. Script que muestre espacio libre del disco

df -h

```
(kali㉿kali)-[~/Desktop]
$ ./ej.sh
Filesystem      Size  Used Avail Use% Mounted on
udev            921M   0    921M   0% /dev
tmpfs           198M  1.1M  197M   1% /run
/dev/sda1       79G   59G   17G  79% /
tmpfs           987M  4.0K  987M   1% /dev/shm
tmpfs           5.0M   0    5.0M   0% /run/lock
tmpfs           987M  364K  987M   1% /tmp
tmpfs           1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           198M  140K  198M   1% /run/user/1000
tmpfs           1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
```

10. Script que cree una carpeta automáticamente

```
GNU nano 8.6                                Desktop/ej.sh *
#!/bin/bash

# Pedir al usuario el nombre de la carpeta
read -p "Ingresa el nombre de la carpeta que deseas crear: " nombre_carpeta

# Comprobar si el usuario escribió algo
if [ -z "$nombre_carpeta" ]; then
    echo "No ingresaste ningún nombre. Saliendo..."
    exit 1
fi

# Crear la carpeta en la ruta actual
if [ -d "$nombre_carpeta" ]; then
    echo "La carpeta '$nombre_carpeta' ya existe."
else
    mkdir "$nombre_carpeta"
    echo "Carpeta '$nombre_carpeta' creada exitosamente."
fi
```

11. Script que cree un archivo con tu nombre y la fecha actual

```
GNU nano 8.6                                Desktop/ej.sh *
#!/bin/bash

# Pedir el nombre del usuario
read -p "Ingresa tu nombre: " nombre

# Comprobar si ingresó algo
if [ -z "$nombre" ]; then
    echo "No ingresaste ningún nombre. Saliendo..."
    exit 1
fi

# Obtener la fecha actual en formato AAAA-MM-DD_HH-MM-SS
fecha=$(date +"%Y-%m-%d_%H-%M-%S")

# Nombre del archivo
archivo="${nombre}_${fecha}.txt"

# Crear el archivo
touch "$archivo"

echo "Archivo '$archivo' creado exitosamente."
```

12. Script que revise si un archivo existe o no

Usar if [-f archivo].

```
GNU nano 8.6
#!/bin/bash

# Pedir al usuario el nombre del archivo
read -p "Ingresa la ruta o nombre del archivo a verificar: " archivo

# Comprobar si se ingresó algo
if [ -z "$archivo" ]; then
    echo "No ingresaste ningún nombre de archivo. Saliendo..."
    exit 1
fi

# Verificar si el archivo existe
if [ -f "$archivo" ]; then
    echo "El archivo '$archivo' existe."
else
    echo "El archivo '$archivo' NO existe."
fi
```

✓ EJERCICIOS INTERMEDIOS (13–24)

13. Script que lea dos números y los sume

```
GNU nano 8.6
#!/bin/bash

# Leer el primer número
read -p "Ingresa el primer número: " num1

# Leer el segundo número
read -p "Ingresa el segundo número: " num2

# Validar que sean números
if ! [[ "$num1" =~ ^-[0-9]+$ ]] || ! [[ "$num2" =~ ^-[0-9]+$ ]]; then
    echo "Error: debes ingresar solo números."
    exit 1
fi

# Sumar los números
suma=$((num1 + num2))

# Mostrar el resultado
echo "La suma de $num1 + $num2 es: $suma"
```

14. Script que compruebe si un servicio está activo

systemctl is-active ssh

```
GNU nano 8.0                                ej.sn *
#!/bin/bash

# Pedir al usuario el nombre del servicio
read -p "Ingresa el nombre del servicio a comprobar: " servicio

# Comprobar que se ingresó algo
if [ -z "$servicio" ]; then
    echo "No ingresaste ningún servicio. Saliendo..."
    exit 1
fi

# Verificar si el servicio existe
if ! systemctl list-units --type=service --all | grep -q "^$servicio.service"; then
    echo "El servicio '$servicio' no existe."
    exit 1
fi

# Comprobar si el servicio está activo
if systemctl is-active --quiet "$servicio"; then
    echo "El servicio '$servicio' está activo."
else
    echo "El servicio '$servicio' NO está activo."
fi
```

15. Script que reinicie automáticamente un servicio si está caído

```
GNU nano 8.0                                ej.sn *
#!/bin/bash

# Pedir al usuario el nombre del servicio
read -p "Ingresa el nombre del servicio a monitorear: " servicio

# Comprobar que se ingresó algo
if [ -z "$servicio" ]; then
    echo "No ingresaste ningún servicio. Saliendo..."
    exit 1
fi

# Verificar si el servicio existe
if ! systemctl list-units --type=service --all | grep -q "^$servicio.service"; then
    echo "El servicio '$servicio' no existe."
    exit 1
fi

# Comprobar si el servicio está activo
if systemctl is-active --quiet "$servicio"; then
    echo "El servicio '$servicio' está activo ✓"
else
    echo "El servicio '$servicio' NO está activo ✗"
    echo "Intentando reiniciarlo..."
    sudo systemctl restart "$servicio"

    # Verificar si se reinició correctamente
    if systemctl is-active --quiet "$servicio"; then
        echo "El servicio '$servicio' se reinició correctamente ✓"
    else
        echo "No se pudo reiniciar el servicio ✗"
    fi
fi
```

16. Script que haga ping a un servidor y avise si falla

```
GNU nano 8.0
#!/bin/bash

# Pedir al usuario la IP o dominio del servidor
read -p "Ingresa la IP o dominio del servidor: " servidor

# Comprobar que se ingresó algo
if [ -z "$servidor" ]; then
    echo "No ingresaste ningún servidor. Saliendo..."
    exit 1
fi

# Hacer ping (1 intento)
ping -c 1 -W 2 "$servidor" > /dev/null 2>&1

# Comprobar el resultado
if [ $? -eq 0 ]; then
    echo "✅ El servidor $servidor está activo."
else
    echo "❌ El servidor $servidor NO responde."
fi
```

17. Script que liste todos los usuarios del sistema

cut -d: -f1 /etc/passwd

```
GNU nano 8.6
cut -d: -f1 /etc/passwd
```

```
(kali㉿kali)-[~]
└─$ sudo Desktop/ej.sh
Filesystem      Size  Used Avail Use% Mounted on
udev            921M   0    921M   0% /dev
tmpfs           198M  1.1M  197M   1% /run
/dev/sda1       79G   59G   17G   79% /
tmpfs           987M  4.0K  987M   1% /dev/shm
tmpfs           5.0M   0    5.0M   0% /run/lock
tmpfs           987M  468K  987M   1% /tmp
tmpfs           1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           198M  136K  198M   1% /run/user/1000
tmpfs           1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
```

18. Script que genere un log en /var/log/local/

```
#!/bin/bash

# Nombre del archivo de log
LOG_DIR="/var/log/local"
LOG_FILE="$LOG_DIR/mi_log.log"

# Comprobar si el directorio existe, si no, crearlo
if [ ! -d "$LOG_DIR" ]; then
    sudo mkdir -p "$LOG_DIR"
    sudo chmod 755 "$LOG_DIR"
    echo "Directorio $LOG_DIR creado."
fi

# Crear o agregar al archivo de log con fecha y mensaje
echo "$(date +%Y-%m-%d %H:%M:%S) - Este es un registro de prueba" | sudo tee -a "$LOG_FILE" > /dev/null

echo "Registro agregado en $LOG_FILE"
```

19. Script que busque archivos mayores de 100MB

```
#!/bin/bash

# Buscar archivos mayores a 100MB en el directorio actual
find . -type f -size +100M
```

20. Script que monitorice RAM cada 10 segundos

```
GNU nano 8.6
#!/bin/bash

# Monitor de RAM cada 10 segundos
echo "Presiona Ctrl+C para salir."
while true; do
    echo "_____ $(date) _____"
    free -h
    sleep 10
done
```

21. Script que cree 10 archivos con nombres secuenciales

```
GNU nano 8.6          ej.sn *
#!/bin/bash

# Crear 10 archivos con nombres secuenciales
for i in {1..10}; do
    touch "archivo$i.txt"
done

echo "Se han creado 10 archivos: archivo1.txt a archivo10.txt"
```

22. Script que elimine archivos temporales automáticamente

```
GNU nano 8.6  
#!/bin/bash  
  
# Eliminar archivos temporales en el directorio actual  
rm -f *.tmp *.temp *~
```

23. Script que comprima una carpeta en [.tar.gz](#)

```
GNU nano 8.6  
#!/bin/bash  
  
# Carpeta a comprimir (primer argumento)  
CARPETA="$1"  
  
# Archivo de salida (segundo argumento)  
SALIDA="$2"  
  
# Comprimir  
tar -czf "$SALIDA" "$CARPETA"  
  
echo "Carpeta $CARPETA comprimida en $SALIDA"
```

24. Script que verifique conexión a Internet

Hacer ping a 8.8.8.8.

```
GNU nano 8.6  
#!/bin/bash  
  
# Hacer ping a 8.8.8.8 una vez  
if ping -c 1 8.8.8.8 &> /dev/null  
then  
    echo "Conexión a Internet: OK"  
else  
    echo "Sin conexión a Internet"  
fi
```

✓ EJERCICIOS AVANZADOS (25–36)

25. Script que haga copia de seguridad de /home

```
GNU nano 8.0  
#!/bin/bash  
  
# Carpeta a respaldar  
ORIGEN="/home"  
  
# Carpeta donde se guardará el backup (por defecto /root/backups)  
DESTINO="${1:-/root/backups}"  
  
# Crear carpeta de destino si no existe  
mkdir -p "$DESTINO"  
  
# Nombre del archivo de backup con fecha  
FECHA=$(date +%Y%m%d_%H%M%S)  
ARCHIVO="$DESTINO/home_backup_$FECHA.tar.gz"  
  
# Crear backup  
tar -czf "$ARCHIVO" "$ORIGEN"  
  
echo "Copia de seguridad creada en $ARCHIVO"
```

26. Script que compruebe si un puerto está abierto

ss -tulnp | grep 22

```
GNU nano 8.0  
#!/bin/bash  
  
# Puerto a comprobar (por defecto 22)  
PUERTO="${1:-22}"  
  
# Buscar puerto  
if ss -tulnp | grep -w ":$PUERTO" &> /dev/null  
then  
    echo "El puerto $PUERTO está abierto."  
else  
    echo "El puerto $PUERTO no está abierto."  
fi
```

27. Script que envíe un mensaje de alerta si un disco supera el 80%

```
#!/bin/bash

# Porcentaje límite
LIMITE=80

# Comprobar el uso de todos los discos
df -h | awk 'NR>1 {print $5 " " $1}' | while read USO DISCO; do
    # Quitar el % del uso
    PORCENTAJE=${USO%\%}
    if [ "$PORCENTAJE" -gt "$LIMITE" ]; then
        echo "ALERTA: El disco $DISCO está al $USO de su capacidad"
    fi
done
```

28. Script con menú interactivo (1. Info – 2. Red – 3. Salir)

```
#!/bin/bash

while true; do
    # Mostrar menú
    echo "==== MENÚ ====="
    echo "1. Info del sistema"
    echo "2. Información de red"
    echo "3. Salir"
    echo "====="
    read -p "Elige una opción [1-3]: " OPCION

    case $OPCION in
        1)
            echo "——— Info del sistema ———"
            uname -a
            df -h
            free -h
            ;;
        2)
            echo "——— Información de red ———"
            ip a
            ss -tulnp
            ;;
        3)
            echo "Saliendo ..."
            exit 0
            ;;
        *)
            echo "Opción inválida. Elige 1, 2 o 3."
            ;;
    esac

    echo "" # Línea en blanco antes de mostrar el menú de nuevo
done
```

29. Script que muestre los 10 archivos más grandes del sistema

```
#!/bin/bash

echo "Buscando los 10 archivos más grandes del sistema, esto puede tardar un poco..."

# Buscar archivos, mostrar tamaño en bytes, ordenar y mostrar los 10 mayores
find / -type f -exec du -h {} + 2>/dev/null | sort -rh | head -n 10
```

30. Script que genere informes diarios de estado

CPU, RAM, disco, red.

```
#!/bin/bash

# Carpeta donde guardar los informes
CARPETA="${1:-/tmp/informes}"
mkdir -p "$CARPETA"

# Nombre del archivo con fecha
FECHA=$(date +%Y%m%d)
ARCHIVO="$CARPETA/informe_-$FECHA.txt"

echo "Generando informe diario del sistema: $ARCHIVO"

{
    echo "==== INFORME DIARIO DEL SISTEMA ====="
    echo "Fecha: $(date)"
    echo ""

    echo "----- CPU -----"
    lscpu
    echo ""

    echo "----- RAM -----"
    free -h
    echo ""

    echo "----- Disco -----"
    df -h
    echo ""

    echo "----- Red -----"
    ip a
    ss -tulnp
    echo ""

    echo "Informe generado correctamente."
} > "$ARCHIVO"

echo "Informe guardado en $ARCHIVO"
```

31. Script que haga un backup incremental

```
#!/bin/bash

# Carpeta origen a respaldar
ORIGEN="${1:-/home/usuario}"

# Carpeta destino del backup
DESTINO="${2:-/backup}"

# Crear carpeta destino si no existe
mkdir -p "$DESTINO"

# Nombre de la carpeta de backup con fecha
FECHA=$(date +%Y%m%d)
DEST_FECHA="$DESTINO/backup_$FECHA"

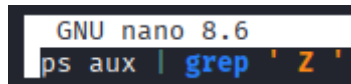
# Hacer backup incremental con rsync
rsync -av --delete --link-dest="$DESTINO/latest" "$ORIGEN/" "$DEST_FECHA/"

# Actualizar enlace 'latest' al backup más reciente
rm -f "$DESTINO/latest"
ln -s "$DEST_FECHA" "$DESTINO/latest"

echo "Backup incremental realizado en $DEST_FECHA"
```

32. Script que detecte procesos zombie

ps aux | grep Z



```
GNU nano 8.6
ps aux | grep Z
```

33. Script que reinicie automáticamente un servicio si falla


```

GNU nano 8.6
#!/bin/bash

# Nombre del servicio a monitorear
SERVICIO="${1:-apache2}"

# Comprobar si el servicio está activo
if systemctl is-active --quiet "$SERVICIO"; then
    echo "El servicio $SERVICIO está funcionando correctamente."
else
    echo "El servicio $SERVICIO no está activo. Reiniciando ..."
    systemctl restart "$SERVICIO"
    if systemctl is-active --quiet "$SERVICIO"; then
        echo "El servicio $SERVICIO se reinició correctamente."
    else
        echo "No se pudo reiniciar el servicio $SERVICIO."
    fi
fi

```

34. Script que registre la IP pública

curl ifconfig.me

```

#!/bin/bash

# Carpeta donde guardar el registro
CARPETA="${1:-/tmp/ip_registros}"
mkdir -p "$CARPETA"

# Archivo de registro con fecha
FECHA=$(date +%Y%m%d)
ARCHIVO="$CARPETA/ip_$FECHA.txt"

# Obtener la IP pública y guardarla con hora
echo "$(date '+%Y-%m-%d %H:%M:%S') - IP Pública: $(curl -s ifconfig.me)" >> "$ARCHIVO"
echo "IP registrada en $ARCHIVO"

```

35. Script que revise logs del sistema y filtre “error”

```

#!/bin/bash

# Archivo de log a revisar (por defecto /var/log/syslog)
LOG="${1:-/var/log/syslog}"

echo "Buscando la palabra 'error' en $LOG ..."

# Filtrar líneas que contengan "error" (sin distinguir mayúsculas/minúsculas)
grep -i "error" "$LOG"

```

36. Programar tu script con cron para que se ejecute cada día

```
GNU nano 8.6 /tmp/crontab.K8A5Tg/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 7 * * * /ruta/completa/a/tu/script/informe_diario.sh >> /ruta/completa/a/tu/script/log_diario.txt 2>&1
```

MINI-PROYECTO DEL MÓDULO 8 — Sistema de Monitorización Completo

Objetivo:

Crear un sistema automático de monitorización realista en Ubuntu.

Debe incluir un script llamado `monitor.sh` que:

✓ Muestre:

- Fecha y hora
- Uso de CPU
- Uso de RAM
- 5 procesos más pesados
- Espacio libre en disco
- Estado de un servicio (apache2 o ssh)
- Conectividad a Internet

✓ Guarde un log en:

`/var/log/monitor.log`

✓ Cree alertas si:

- CPU > 80%
- RAM > 80%
- Disco > 80%

✓ Se ejecute automáticamente con cron cada 15 minutos:

```
*/15 * * * * /ruta/monitor.sh
```

✓ Incluya documentación técnica del proyecto:

- Código usado
- Explicación
- Capturas
- Solución a errores

```
GNU nano 2.9.3
#!/bin/bash

# Monitor.sh - Script de monitorización automática

# Archivo de log
LOG_FILE="/var/log/monitor.log"

# Función para escribir en log con fecha
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$LOG_FILE"
}

# — Monitorización —

# 1. Fecha y hora
DATE_TIME=$(date '+%Y-%m-%d %H:%M:%S')
log "Fecha y hora: $DATE_TIME"

# 2. Uso de CPU
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8"%"}')
log "Uso de CPU: $CPU_USAGE"

# 3. Uso de RAM
RAM_TOTAL=$(free -m | awk '/Mem:/ {print $2}')
RAM_USED=$(free -m | awk '/Mem:/ {print $3}')
RAM_PERCENT=$(( RAM_USED * 100 / RAM_TOTAL ))
log "Uso de RAM: $RAM_USED MB / $RAM_TOTAL MB ($RAM_PERCENT%)"

# 4. 5 procesos más pesados
log "5 procesos más pesados:"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 6 >> "$LOG_FILE"

# 5. Espacio libre en disco
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}')
log "Uso de disco en /: $DISK_USAGE"

# 6. Estado de un servicio (apache2 o ssh)
SERVICE="apache2" # Cambia a ssh si prefieres
SERVICE_STATUS=$(systemctl is-active $SERVICE)
log "Estado del servicio $SERVICE: $SERVICE_STATUS"
```

```
# 7. Conectividad a Internet
ping -c 1 8.8.8.8 &> /dev/null
if [ $? -eq 0 ]; then
    INTERNET_STATUS="Conectado"
else
    INTERNET_STATUS="No conectado"
fi
log "Conectividad a Internet: $INTERNET_STATUS"

# — Alertas —
if [ ${CPU_USAGE%%}% -gt 80 ]; then
    log "ALERTA: Uso de CPU mayor al 80%!"
fi

if [ $RAM_PERCENT -gt 80 ]; then
    log "ALERTA: Uso de RAM mayor al 80%!"
fi

DISK_PERCENT=${DISK_USAGE%%}%
if [ $DISK_PERCENT -gt 80 ]; then
    log "ALERTA: Uso de disco mayor al 80%!"
fi

log "_____"
```